TinyMPC with Differential Flatness: α-Parameterized Templates for Efficient Quadrotor Control

Jianghan Zhang
Dartmouth College

***Abstract*—TinyMPC enables real-time model-predictive control on microcontrollers by caching the infinite-horizon Riccati solution and requiring only matrix–vector products at each ADMM iteration. However, its linearization is fixed at a single operating point (typically hover), which degrades tracking performance for aggressive trajectories. We exploit the *differential flatness* of quadrotor dynamics to reparameterize the linearization by a flat parameter $\boldsymbol{\alpha} = (a_x, a_y, a_z, \psi)$. This yields closed-form equilibria and Jacobians—no automatic differentiation required—making the computation FPGA-portable. We present three extensions of TinyMPC: (i) *Flat Template MPC*, which precomputes DARE caches at a grid of α values for online nearest-neighbor lookup; (ii) *Analytic Template*, a zero-iteration state-feedback controller using first-order gain interpolation ($\sim$260 arithmetic operations per control output); and (iii) *Barrier-Augmented Template*, which absorbs log-barrier constraint information into the DARE cache so that the zero-iteration controller is inherently constraint-aware, with optional Newton–IPM refinement at quadratic convergence. Preliminary Python simulations on a Crazyflie-scale quadrotor show that, on figure-eight trajectory tracking, the Flat Template achieves tracking accuracy within 6% of online relinearization (0.035 m average position error) at $29\times$ lower wall-clock time, while the Analytic Template maintains 0.036 m error with zero ADMM iterations.**

I. Introduction

Model-predictive control (MPC) is a powerful framework for controlling highly dynamic robots subject to state and input constraints [?]. TinyMPC [?] demonstrated that, by caching the infinite-horizon discrete algebraic Riccati equation (DARE) solution offline and performing only matrix–vector products online, the ADMM-based MPC solver can run on microcontrollers with as little as 192 kB of RAM at rates exceeding 500 Hz.

A key assumption in TinyMPC is that the dynamics are linearized at a *fixed operating point*—typically the hover equilibrium for a quadrotor. This is reasonable for near-hover regulation but becomes a limitation for aggressive trajectory tracking, where the true linearization varies significantly along the reference. The standard remedy—online relinearization at each timestep—requires Jacobian evaluation (via automatic differentiation or finite differences) and a fresh DARE solve, both of which are computationally expensive and ill-suited for resource-constrained platforms.

We observe that quadrotor dynamics are *differentially flat* with flat output $\boldsymbol{y} = (p_x, p_y, p_z, \psi)$ [?], [?]. This classical property has been widely used for trajectory generation [?], [?], but its implications for *MPC solver structure* have received less attention. We show that flatness enables a closed-form parameterization of both the equilibrium and the Jacobian by a single four-dimensional *flat parameter* $\boldsymbol{\alpha} = (a_x, a_y, a_z, \psi) \in \mathbb{R}^4$, representing the commanded translational acceleration and yaw angle. Because the resulting expressions involve only elementary trigonometric and algebraic operations, they require no automatic differentiation and are directly implementable on FPGAs.

Crucially, because $\boldsymbol{\alpha}$ is four-dimensional—compared to the $n = 12$ dimensional state—it serves as a natural *scheduling variable* for the DARE solution. This connects our approach to classical gain scheduling [?] and linear parameter-varying (LPV) control [?], but with the scheduling space compressed from $\mathbb{R}^n$ to $\mathbb{R}^4$ by the flat map. We make this connection precise in Section **??** and show that the Analytic Template is equivalent to a first-order gain schedule over $\boldsymbol{\alpha}$, where the gain sensitivity is obtained from perturbation analysis of the DARE [?], [?].

Building on this observation, we propose three extensions of TinyMPC:

1) **Flat Template MPC** (Section **??**): Precompute DARE caches at a grid of $\boldsymbol{\alpha}$ values offline. Online, look up the nearest cache and run K_{ADMM} ADMM iterations as in standard TinyMPC, but with $\boldsymbol{\alpha}$-specific matrices $(A_d, B_d, K_\infty, P_\infty, C_1, C_2)$.
2) **Analytic Template** (Section **??**): Precompute the gain K_0 at hover and its four partial derivatives $\partial K/\partial\alpha_i$ offline. Online, interpolate $K(\boldsymbol{\alpha}) \approx K_0 + \sum_i \delta\alpha_i\, \partial_i K$ and apply a single feedback step—no ADMM iterations, no constraint projection. The total cost is $\sim$260 multiply-accumulate (MAC) operations.
3) **Barrier-Augmented Template** (Section **??**): Absorb log-barrier constraint penalties into the DARE cache, so that the zero-iteration controller inherently avoids actuator limits. Optional Newton–IPM refinement online yields hard constraint satisfaction at quadratic convergence.

All flat-parameterized variants shift the coordinate frame to the flat equilibrium $(x^*(\boldsymbol{\alpha}), u^*(\boldsymbol{\alpha}))$, so the solver always operates in a neighborhood of zero error—regardless of the trajectory aggressiveness. Fig. **??** illustrates the pipeline.

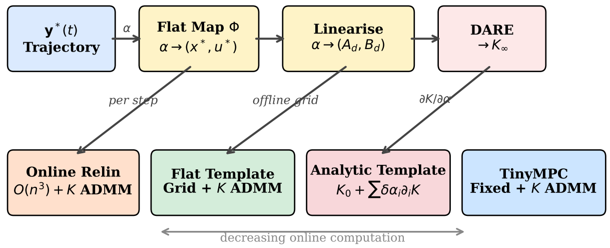

Fig. 1. Controller hierarchy. The flat map Φ and analytic linearization (yellow) are shared by all flat-parameterized variants. Moving right trades online computation for approximation: Online Relinearization re-solves DARE per step; Flat Template looks up a precomputed cache; the Analytic Template interpolates $K(\boldsymbol{\alpha})$ in ∼260 ops.

Contributions

- An explicit, autograd-free linearization of quadrotor dynamics parameterized by $\boldsymbol{\alpha} \in \mathbb{R}^4$ (Section **??**).
- Flat Template MPC: an $\boldsymbol{\alpha}$-indexed cache family for TinyMPC's ADMM solver (Section **??**).
- Analytic Template: a zero-iteration controller with first-order gain approximation (Section **??**).
- Barrier-Augmented Template: absorbing log-barrier constraint information into the DARE cache for zero-iteration constraint-aware control, with optional IPM refinement at quadratic convergence (Section **??**).
- Convergence analysis: quadratic gain approximation error from flatness-compressed scheduling, and exponential time-contraction with explicit rate–conservatism tradeoff (Section **??**).
- Preliminary numerical validation comparing four controller variants on hover, figure-eight, and constrained circle tracking (Section **??**).

II. Background

A. Mathematical Preliminaries

This section introduces the three mathematical tools that the paper builds on: ADMM, the discrete algebraic Riccati equation, and the log-barrier interior-point method. Readers familiar with these topics may skip to Section **??**.

ADMM (Alternating Direction Method of Multipliers). Many MPC problems take the form

$$\min_z \ f(z) \quad \text{s.t.} \quad z \in \mathcal{C}, \tag{1}$$

where f is a smooth (often quadratic) cost and $\mathcal{C}$ encodes constraints (e.g., actuator limits, state bounds). ADMM [**?**] splits this into two alternating steps by introducing an auxiliary variable $\tilde{z}$ and a dual variable λ:

(a) *Minimize* f without constraints (smooth, often admits a closed-form or Riccati-based solution): $z^{k+1} = \arg\min_z \ f(z) + \frac{\rho}{2}\|z - \tilde{z}^k + \lambda^k\|^2$.
(b) *Project* onto constraints (often a simple clipping operation): $\tilde{z}^{k+1} = \mathrm{proj}_{\mathcal{C}}(z^{k+1} + \lambda^k)$.
(c) *Update* the dual variable: $\lambda^{k+1} = \lambda^k + z^{k+1} - \tilde{z}^{k+1}$.

The penalty $\rho > 0$ controls the coupling strength between the smooth and constraint steps. ADMM converges at a *linear* rate: the primal residual $\|z - \tilde{z}\|$ decreases geometrically with the number of iterations [**?**]. For MPC, the key property is that step (a) has the structure of an unconstrained LQR problem, solvable by a Riccati recursion, while step (b) is a componentwise clipping operation. This separation is what enables the DARE-caching strategy of TinyMPC.

Discrete Algebraic Riccati Equation (DARE). Consider the infinite-horizon LQR problem for the linear system $x_{k+1} = Ax_k + Bu_k$ with stage cost $\ell(x,u) = \frac{1}{2}(x^\top Q x + u^\top R u)$, where $Q \succeq 0$ and $R \succ 0$. The optimal cost-to-go from state x is $V^*(x) = \frac{1}{2}x^\top P x$, where P satisfies the DARE:

$$P = Q + A^\top P A - A^\top P B (R + B^\top P B)^{-1} B^\top P A. \tag{2}$$

The optimal feedback gain is $K = (R + B^\top P B)^{-1} B^\top P A$, so that $u^* = -Kx$ minimizes the infinite-horizon cost. The closed-loop matrix $A_{\mathrm{cl}} = A - BK$ is stable: all eigenvalues lie strictly inside the unit circle, guaranteeing exponential convergence $\|x_k\| \le c\,\bar{\rho}(A_{\mathrm{cl}})^k \|x_0\|$ where $\bar{\rho}(A_{\mathrm{cl}}) < 1$ is the spectral radius.

The DARE is solved offline by iterating the Riccati recursion $P_k = Q + A^\top P_{k+1} A - A^\top P_{k+1} B (R + B^\top P_{k+1} B)^{-1} B^\top P_{k+1} A$ backward from a terminal condition P_N until convergence. The converged P and K encode the entire infinite-horizon optimal controller in two matrices.

Log-Barrier Interior-Point Method. To handle inequality constraints $g_j(u) \le 0$, $j = 1, \ldots, p$, the log-barrier method [**?**] replaces them with a smooth penalty:

$$\min_u \ f(u) - \mu \sum_{j=1}^{p} \log(-g_j(u)), \tag{3}$$

where $\mu > 0$ is the barrier parameter. The logarithmic term is finite only when all constraints are strictly satisfied ($g_j < 0$), and tends to $+\infty$ as any constraint approaches its boundary ($g_j \to 0^-$). As $\mu \to 0$, the solution of (**??**) approaches the solution of the original constrained problem.

For box constraints $u_{\min} \le u \le u_{\max}$, $g_j(u) = u_j - u_{\max,j}$ and $g_{j+m}(u) = u_{\min,j} - u_j$, giving the barrier $-\mu[\log(u_j - u_{\min,j}) + \log(u_{\max,j} - u_j)]$. The Hessian (second derivative) of this barrier is a diagonal matrix whose entries are large near the constraint boundaries and small far from them.

Each subproblem (**??**) is smooth and unconstrained (in the strict interior), so it can be solved by Newton's method. Newton's method converges *quadratically*: the error decreases as $\|e_{k+1}\| \le C\|e_k\|^2$, meaning that 2–3 iterations typically reduce a moderate initial error to machine precision [**?**]. This is in contrast to ADMM's linear convergence, where a fixed fraction of the error is removed per iteration.

B. Related Work

Gain scheduling. The classical approach to controlling nonlinear systems around varying operating points is gain scheduling [**?**]: solve the linear control problem (e.g., LQR or H_∞) at a grid of operating points offline, then interpolate the resulting gains online. Rugh and Shamma [?] survey the extensive theory, including conditions under which interpolated controllers inherit stability from the frozen-parameter designs. Our Flat Template MPC is a gain-scheduled ADMM-MPC controller where the scheduling variable is the flat parameter $\boldsymbol{\alpha}$, and our Analytic Template is a first-order gain schedule with an explicit interpolation formula (**??**).

LPV control. Linear parameter-varying (LPV) systems generalize gain scheduling by treating the varying linearization as a parameter-dependent plant $\dot{x} = A(\theta)x + B(\theta)u$ where $\theta(t)$ is measurable in real time [?]. Our framework fits this structure with $\theta = \boldsymbol{\alpha}$, but we exploit flatness to guarantee that $\boldsymbol{\alpha}$ is a *four-dimensional sufficient statistic* for the equilibrium—rather than requiring the full n-dimensional state as scheduling variable.

Riccati sensitivity analysis. The dependence of the DARE solution on system parameters has been studied since Hewer [?], with explicit perturbation bounds developed by Konstantinov et al. [?]. Our gain sensitivities $\partial K/\partial \alpha_i$ in (**??**) are a numerical instance of this theory, approximated by central differences. The key practical insight is that for quadrotor dynamics, $K(\boldsymbol{\alpha})$ varies smoothly and slowly with $\boldsymbol{\alpha}$, so a first-order expansion suffices (4% mean relative error over the tested range).

Flatness-based tracking control. Differential flatness has been widely used for quadrotor trajectory generation [?], [?] and feedforward computation [?]. Mellinger and Kumar [?] use the flat map to compute reference states and inputs, then track with a fixed-gain controller. Our approach extends this by additionally using the flat map to parameterize the *feedback gain itself*, adapting K to the current operating condition through the DARE sensitivity.

Explicit MPC. An alternative to online optimization is explicit MPC [?], which precomputes the optimal control law as a piecewise-affine function of the state via multi-parametric programming. The Flat Template MPC shares the "precompute offline, look up online" philosophy, but avoids the exponential complexity of explicit MPC by storing only DARE caches (not full polyhedral partitions) and running a small number of ADMM iterations online.

C. TinyMPC

TinyMPC [?] solves the constrained linear MPC problem, formulated as a quadratic program (QP) [?], [?],

$$\begin{aligned} \min_{x_{1:N}, u_{1:N-1}} \quad & \tfrac{1}{2} x_N^\top Q_f x_N + \sum_{k=1}^{N-1} \left(\tfrac{1}{2} x_k^\top Q x_k + \tfrac{1}{2} u_k^\top R u_k \right) \\ \text{s.t.} \quad & x_{k+1} = A x_k + B u_k, \\ & x_k \in \mathcal{X}, \quad u_k \in \mathcal{U}, \end{aligned} \tag{4}$$

via ADMM, where $\rho > 0$ is the ADMM penalty parameter. The key insight is that, for a sufficiently long horizon N, the backward Riccati recursion converges to a steady-state solution (K_∞, P_∞). The matrix P_∞ is the infinite-horizon optimal cost-to-go: it compresses the effect of all future stages into a single $n \times n$ matrix, so that $\frac{1}{2}\delta x^\top P_\infty \, \delta x$ approximates the minimum cost from state δx to the origin over an infinite horizon. TinyMPC solves the discrete algebraic Riccati equation (DARE) for P_∞ *offline* and caches the derived quantities:

$$\begin{aligned} K_\infty &= (R_\rho + B^\top P_\infty B)^{-1} B^\top P_\infty A, \\ C_1 &= (R_\rho + B^\top P_\infty B)^{-1}, \\ C_2 &= (A - B K_\infty)^\top, \end{aligned} \tag{5}$$

where $R_\rho = R + \rho I$ is the ADMM-augmented input cost and $Q_\rho = Q + \rho I$ is the ADMM-augmented state cost. The gain K_∞ is the optimal LQR gain for the augmented problem; C_1 and C_2 are auxiliary matrices that eliminate all matrix inversions from the online computation. Each ADMM iteration then requires only matrix–vector products:

(i) *Backward pass*: compute feedforward corrections $d_k = C_1(B^\top p_{k+1} + r_k)$, $p_k = q_k + C_2\, p_{k+1} - K_\infty^\top r_k$.
(ii) *Forward pass*: $u_k = -K_\infty x_k - d_k$, $x_{k+1} = A x_k + B u_k$.
(iii) *Slack update*: project onto constraints.
(iv) *Dual update*: gradient ascent on multipliers.

The entire online computation is *matrix-inversion free*. However, the matrices A, B (and hence all cached quantities) are fixed at a single linearization point.

D. Differential Flatness of Quadrotor Dynamics

A quadrotor with state $(p, \dot{p}, R, \omega) \in \mathbb{R}^3 \times \mathbb{R}^3 \times \mathrm{SO}(3) \times \mathbb{R}^3$ and collective thrust T along the body z-axis satisfies

$$m\ddot{p} = T R e_3 - m g e_3. \tag{6}$$

The system is differentially flat with flat output $\boldsymbol{y} = (p_x, p_y, p_z, \psi)$ [?], [?]. This means that every state and input can be expressed as an algebraic function of $\boldsymbol{y}$ and a finite number of its time derivatives.

The flat map proceeds as follows. Define the *total acceleration vector*

$$\boldsymbol{t} = \ddot{p} + g e_3 = \begin{pmatrix} a_x \\ a_y \\ a_z + g \end{pmatrix}, \tag{7}$$

where (a_x, a_y, a_z) is the commanded translational acceleration in the world frame. Then:

1) **Thrust:** $T = m\|\boldsymbol{t}\|$.
2) **Attitude:** The body z-axis is $z_B = \boldsymbol{t}/\|\boldsymbol{t}\|$. Given yaw ψ, the rotation R is fully determined. In Euler angles (ZYX convention):

$$\phi^* = \arcsin\!\big((a_x \sin\psi - a_y \cos\psi)/\|\boldsymbol{t}\|\big), \tag{8}$$
$$\theta^* = \arctan 2\big(a_x \cos\psi + a_y \sin\psi,\ a_z + g\big). \tag{9}$$

3) **Angular velocity and torques:** determined by $p^{(3)}$ and $p^{(4)}$ (jerk and snap) through the kinematic and Euler equations; at equilibrium, $\omega^* = 0$.

We define the *flat parameter* $\boldsymbol{\alpha} = (a_x, a_y, a_z, \psi) \in \mathbb{R}^4$, which fully specifies the equilibrium state and input:

$$\Phi : \boldsymbol{\alpha} \ \mapsto\ (x^*(\boldsymbol{\alpha}),\ u^*(\boldsymbol{\alpha})). \tag{10}$$

III. Method

A. Analytic Linearization

Given $\boldsymbol{\alpha}$, we construct the continuous-time Jacobians $A_c(\boldsymbol{\alpha}), B_c(\boldsymbol{\alpha})$ of the 12-dimensional Euler-angle state $x = (p, \phi, \theta, \psi, v, \omega)$ about the flat equilibrium (**??**).

The 12×12 matrix A_c has the block structure

$$A_c = \begin{pmatrix} 0_{3\times3} & 0_{3\times3} & I_3 & 0_{3\times3} \\ 0_{3\times3} & 0_{3\times3} & 0_{3\times3} & W(\phi^*, \theta^*) \\ 0_{3\times3} & G(\boldsymbol{\alpha}) & 0_{3\times3} & 0_{3\times3} \\ 0_{3\times3} & 0_{3\times3} & 0_{3\times3} & 0_{3\times3} \end{pmatrix}, \qquad (11)$$

where $G(\boldsymbol{\alpha}) \in \mathbb{R}^{3\times3}$ is the gravity coupling matrix (derivative of the thrust vector with respect to attitude at equilibrium) and $W(\phi^*, \theta^*)$ is the Euler-angle kinematic map relating body angular velocity to Euler rates.

The 12×4 input matrix B_c maps the four motor thrusts to state derivatives. It has nonzero blocks in the velocity rows (thrust direction $\times k_t/m$, where k_t is the per-motor thrust coefficient and m is the quadrotor mass) and the angular velocity rows ($J^{-1}M_{\text{mix}}$, where $J \in \mathbb{R}^{3\times3}$ is the body-frame inertia tensor and $M_{\text{mix}} \in \mathbb{R}^{3\times4}$ is the motor mixing matrix that maps individual motor thrusts to body torques via the arm geometry).

Discretization via forward Euler yields

$$A_d = I_{12} + A_c \Delta t, \qquad B_d = B_c \Delta t. \qquad (12)$$

The entire computation is closed-form trigonometry and linear algebra, requiring no automatic differentiation.

B. Flat Template MPC

Flat Template MPC extends TinyMPC by maintaining a *family* of DARE caches indexed by $\boldsymbol{\alpha}$.

Offline. Select a grid $\{\boldsymbol{\alpha}^{(j)}\}_{j=1}^{M}$ (e.g., sampled along the reference trajectory). For each grid point:

1) Compute $(A_d^{(j)}, B_d^{(j)}) = (\boldsymbol{\alpha}^{(j)})$.
2) Solve the standard (non-augmented) discrete LQR to obtain the terminal cost matrix $P_{\text{lqr}}^{(j)}$.
3) Solve augmented DARE (**??**) to obtain $K_\infty^{(j)}, P_\infty^{(j)}, C_1^{(j)}, C_2^{(j)}$.
4) Store the full cache $\{A_d, B_d, K_\infty, P_\infty, C_1, C_2\}^{(j)}$.

Online. At each control step with current flat parameter $\boldsymbol{\alpha}$:

1) Look up the nearest cache entry $j^* = \arg\min_j \|\boldsymbol{\alpha} - \boldsymbol{\alpha}^{(j)}\|$.
2) Compute the flat equilibrium $(x^*, u^*) = \Phi(\boldsymbol{\alpha})$.
3) Form the error state $\delta x = x - x^*$.
4) Run K_{ADMM} ADMM iterations using cache j^* (identical to TinyMPC's backward/forward/slack/dual passes).
5) Apply $u = u^* + \delta u_0$, where δu_0 is the first-step control from the ADMM solution.

This preserves TinyMPC's matrix-inversion-free online structure while adapting the linearization to the current operating point. The offline cost is M DARE solves (each ∼5000 Riccati iterations), and the online lookup is $O(M)$ nearest-neighbor search.

C. Analytic Template

The Analytic Template eliminates ADMM iterations entirely, producing a control output in ∼260 arithmetic operations.

Offline.

1) Set $\boldsymbol{\alpha}_0 = (0, 0, 0, 0)$ (hover).
2) Solve the augmented DARE at $\boldsymbol{\alpha}_0$ to obtain the base gain $K_0 \in \mathbb{R}^{4\times12}$.
3) For $i = 1, \ldots, 4$, compute gain sensitivities via central differences:
$$\frac{\partial K}{\partial \alpha_i} \approx \frac{K(\boldsymbol{\alpha}_0 + \epsilon\, e_i) - K(\boldsymbol{\alpha}_0 - \epsilon\, e_i)}{2\epsilon}. \qquad (13)$$
4) Store K_0 and $\{\partial_i K\}_{i=1}^4$: a total of $5 \times 4 \times 12 = 240$ floating-point numbers.

Online. Given state x and flat parameter $\boldsymbol{\alpha}$:

1) $\delta\boldsymbol{\alpha} = \boldsymbol{\alpha} - \boldsymbol{\alpha}_0$.
2) Compute $(x^*, u^*) = \Phi(\boldsymbol{\alpha})$.
3) Interpolate the gain (192 multiply-accumulate operations):
$$K(\boldsymbol{\alpha}) \approx K_0 + \sum_{i=1}^{4} \delta\alpha_i \, \frac{\partial K}{\partial \alpha_i}. \qquad (14)$$
4) Compute the control (48 MACs):
$$u = u^*(\boldsymbol{\alpha}) - K(\boldsymbol{\alpha})\,(x - x^*(\boldsymbol{\alpha})). \qquad (15)$$
5) (Optional) Clip to actuator bounds.

Relationship to TinyMPC. The Analytic Template output is *exactly equal* to the first forward-pass step of TinyMPC's ADMM iteration with zero warm-start (i.e., before any constraint information is incorporated). Specifically, when all ADMM dual and slack variables are initialized to zero, the first-iteration forward pass produces $u_0 = -K_\infty\, \delta x$, which matches (**??**) since $K(\boldsymbol{\alpha}) \approx K_\infty(\boldsymbol{\alpha})$ from the same augmented DARE. Subsequent ADMM iterations incorporate constraint information through the slack and dual updates. Thus, the Analytic Template trades constraint handling for extreme computational efficiency.

Connection to gain scheduling and Riccati sensitivity. The control law (**??**) with the gain interpolation (**??**) is precisely a *first-order gain schedule* over the flat parameter $\boldsymbol{\alpha}$ [**?**]. The gain sensitivities $\partial K/\partial\alpha_i$ are instances of the classical Riccati perturbation theory [**?**], [**?**], evaluated numerically via (**??**). What makes this gain schedule unusually compact is differential flatness: whereas a generic gain schedule over the n-dimensional state would require $O(n)$ sensitivity matrices, flatness reduces the scheduling space to $\mathbb{R}^4$, yielding exactly four sensitivity matrices regardless of state dimension. This produces the identity:

$$\underbrace{u^* - K(\boldsymbol{\alpha})\,\delta x}_{\text{Analytic Template}} \;=\; \underbrace{u^* - K_\infty(\boldsymbol{\alpha})\,\delta x}_{\text{ADMM iter. 0}} + O(\epsilon^2), \qquad (16)$$

where the $O(\epsilon^2)$ term is the gain approximation error from the first-order Taylor expansion.

D. Barrier-Augmented Template

The Analytic Template handles constraints only by clipping—the gain $K(\boldsymbol{\alpha})$ is unaware of actuator limits. We now show that log-barrier functions can be absorbed into the DARE cache, yielding a zero-iteration controller that is inherently constraint-aware.

Interior-point formulation. Consider box constraints $u_{\min} \le u \le u_{\max}$. The log-barrier method [?], [?] replaces these with a smooth penalty:

$$\mathcal{B}_\mu(u) = -\mu \sum_{j=1}^{m} \big[\log(u_j - u_{\min,j}) + \log(u_{\max,j} - u_j)\big], \quad (17)$$

where $\mu > 0$ is the barrier parameter controlling the penalty sharpness.

Barrier Hessian at equilibrium. The key observation is that the barrier's second derivative (Hessian) at a given operating point is a *known diagonal matrix* that depends only on the distance from each input channel to its bounds. At the flat equilibrium $u^*(\boldsymbol{\alpha})$, this Hessian is:

$$H_\mu(\boldsymbol{\alpha}) = \mu \cdot \operatorname{diag}\left(\frac{1}{(u_j^* - u_{\min,j})^2} + \frac{1}{(u_{\max,j} - u_j^*)^2} \right). \quad (18)$$

This matrix is large when $u^*(\boldsymbol{\alpha})$ is near a constraint boundary and small when it is far from all boundaries—exactly the structure needed to make the controller more conservative near limits.

State barrier Hessian. For state constraints $x_{\min} \le x \le x_{\max}$ (e.g., position bounds $|p_i| \le L$), the same log-barrier construction yields a state barrier Hessian at the equilibrium $x^*(\boldsymbol{\alpha})$:

$$H_{x,\mu}(\boldsymbol{\alpha}) = \mu \cdot \operatorname{diag}\left(\frac{1}{(x_i^* - x_{\min,i})^2} + \frac{1}{(x_{\max,i} - x_i^*)^2} \right), \quad (19)$$

applied only to the constrained state dimensions (unconstrained dimensions contribute zero). In error coordinates where $x^*_{\text{pos}} = 0$, the equilibrium position equals the reference, so the barrier depends on the distance from the reference to the constraint boundary: $[H_{x,\mu}]_{i,i} = \mu\big(1/(p^*_{\text{ref},i} + L)^2 + 1/(L - p^*_{\text{ref},i})^2\big)$. This is $\boldsymbol{\alpha}$-dependent through the reference trajectory, making the barrier naturally time-varying.

Barrier-augmented DARE. Define the barrier-augmented costs:

$$R_\mu(\boldsymbol{\alpha}) = R + H_\mu(\boldsymbol{\alpha}), \quad (20)$$

$$Q_\mu(\boldsymbol{\alpha}) = Q_{\text{lqr}} + H_{x,\mu}(\boldsymbol{\alpha}), \quad (21)$$

where Q_{lqr} is the terminal cost from the standard (non-augmented) discrete LQR. Solving the augmented DARE with both R_μ and Q_μ:

$$P_\mu = Q_{\mu,\rho} + A_d^\top P_\mu \big(A_d - B_d\, G_\mu^{-1}\, B_d^\top P_\mu A_d\big), \quad (22)$$

where $Q_{\mu,\rho} = Q_\mu + \rho I$ and $G_\mu = R_\mu + \rho I + B_d^\top P_\mu B_d$, yields the barrier-augmented gain $K_\mu(\boldsymbol{\alpha}) = G_\mu^{-1} B_d^\top P_\mu A_d$. This gain inherently reduces authority on input channels that

Algorithm 1 Barrier-Augmented Template MPC

Require: Constraint bounds $u_{\min}, u_{\max}$, $x_{\min}, x_{\max}$ (optional), barrier parameter μ

1: — *Offline* —
2: Set $\boldsymbol{\alpha}_0 = (0,0,0,0)$ (hover)
3: $(x^*, u^*) \leftarrow \Phi(\boldsymbol{\alpha}_0)$ *flat equilibrium*
4: $H_\mu \leftarrow$ (**??**) from u^* and input bounds
5: $H_{x,\mu} \leftarrow$ (**??**) from x^* and state bounds
6: $R_\mu \leftarrow R + H_\mu$; $Q_\mu \leftarrow Q_{\text{lqr}} + H_{x,\mu}$ *barrier-augmented costs*
7: Solve DARE (**??**) with $R_\mu, Q_\mu \to K_{\mu,0}$
8: **for** $i = 1, \ldots, 4$ **do**
9: $\quad K_\mu^+ \leftarrow$ solve DARE at $\boldsymbol{\alpha}_0 + \epsilon\, e_i$; $K_\mu^- \leftarrow$ at $\boldsymbol{\alpha}_0 - \epsilon\, e_i$
10: $\quad \frac{\partial K_\mu}{\partial \alpha_i} \leftarrow (K_\mu^+ - K_\mu^-)/(2\epsilon)$ *central differences*
11: **end for**
12: — *Online (per control step)* —
13: $\boldsymbol{\alpha} \leftarrow$ current flat parameter
14: $K_\mu(\boldsymbol{\alpha}) \approx K_{\mu,0} + \sum_i \delta\alpha_i\, \partial_i K_\mu$ *interpolate*
15: $(x^*, u^*) \leftarrow \Phi(\boldsymbol{\alpha})$
16: $u \leftarrow u^* - K_\mu(\boldsymbol{\alpha})\,(x - x^*)$ $\sim$260 *ops*
17: — *Optional: IPM refinement* —
18: **for** $k = 1, \ldots, K_{\text{IPM}}$ **do**
19: $\quad H_\mu \leftarrow$ (**??**) at current $u^{(k-1)}$
20: $\quad$ Riccati pass with updated $R_\mu \to \delta u$
21: $\quad u^{(k)} \leftarrow u^{(k-1)} + \eta\, \delta u$ *Newton step*
22: **end for**

are near their limits and increases the effective state cost near constraint boundaries: when the reference position approaches the square bound, $[H_{x,\mu}]_{i,i}$ grows, penalizing state excursions in that direction and making the controller more conservative precisely where constraints are tightest.

The offline and online procedures are given in Algorithm **??**.

α-dependent conservatism. The barrier Hessian (**??**) depends on $\boldsymbol{\alpha}$ through $u^*(\boldsymbol{\alpha})$. At hover ($\boldsymbol{\alpha} = 0$), the four motors produce equal thrust at midrange, and H_μ is moderate in all channels. At aggressive maneuvers (large $\|a\|$), some motors approach saturation and H_μ becomes large in those channels, automatically increasing conservatism precisely where constraints are tightest. The scheduling variable $\boldsymbol{\alpha}$ thus simultaneously parameterizes the linearization, the equilibrium, *and* the constraint tightness—all through a single four-dimensional quantity.

Online IPM refinement. For tighter constraint satisfaction, K_{IPM} Newton steps can be performed online (lines 14–18 of Algorithm **??**). The initial iterate $u^{(0)}$ is the output of line 12—the barrier-augmented template's zero-iteration output, which is already near-feasible by construction. Each Newton step recomputes the barrier Hessian (**??**) at the current iterate $u^{(k-1)}$ (not at the equilibrium) and solves one Riccati backward pass with the updated R_μ—the same $O(n^2)$ cost per step as an ADMM iteration. Unlike ADMM (linear convergence), the Newton–IPM iteration converges *quadratically* near the solution [?], [?]: for moderate constraint violations (the warm start is already near-feasible), 2–3 iterations typically reduce the KKT residual below solver tolerance.

Extended equivalence chain. The five variants now form a complete hierarchy:

$$\text{Online Relin} \xrightarrow{\text{cache}} \text{Flat Template} \xrightarrow{K\to 0}$$
$$\text{Barrier Templ.} \xrightarrow{\mu\to 0} \text{Analytic Templ.} \xrightarrow{\text{1st}} \text{Gain Sched.} \quad (23)$$

The Barrier Template sits between the Flat Template (hard constraints via K ADMM iterations) and the Analytic Template (no constraints, clip only): it provides soft constraint satisfaction at zero online iterations, with optional IPM refinement for hard guarantees. Setting $\mu = 0$ recovers the unconstrained Analytic Template; increasing μ trades tracking aggressiveness for constraint margin.

IV. Convergence Analysis

We now formalize the two key convergence properties that differential flatness enables: (i) a quadratic gain approximation error and (ii) exponential time-contraction of the closed-loop system.

Proposition 1 (Quadratic gain approximation)**.** *Let $K(\boldsymbol{\alpha})$ denote the DARE gain* (**??**) *at flat parameter $\boldsymbol{\alpha}$, and let $\tilde{K}(\boldsymbol{\alpha}) = K_0 + \sum_{i=1}^{4}(\alpha_i - \alpha_{0,i})\,\partial_i K$ be the first-order approximation* (**??**)*. Suppose $K(\cdot)$ is twice continuously differentiable in a neighborhood of $\boldsymbol{\alpha}_0$ (guaranteed when $(A_d(\boldsymbol{\alpha}), B_d(\boldsymbol{\alpha}))$ is stabilizable and $(Q^{1/2}, A_d(\boldsymbol{\alpha}))$ is detectable [?]). Then*

$$\|K(\boldsymbol{\alpha}) - \tilde{K}(\boldsymbol{\alpha})\|_F \;\le\; \tfrac{1}{2}\, L_K\, \|\boldsymbol{\alpha} - \boldsymbol{\alpha}_0\|^2, \qquad (24)$$

where $L_K = \max_i \sup_{\boldsymbol{\alpha}} \|\partial^2 K / \partial\alpha_i \partial\alpha_j\|_F$ is the Lipschitz constant of the gain Jacobian.

Proof. This is the standard second-order Taylor remainder applied to the matrix-valued map $\boldsymbol{\alpha} \mapsto K(\boldsymbol{\alpha}) \in \mathbb{R}^{m\times n}$. The twice-differentiability of K with respect to the system parameters (A, B) follows from the implicit function theorem applied to the DARE [?], [?], composed with the smooth (trigonometric) dependence of $(A_d(\boldsymbol{\alpha}), B_d(\boldsymbol{\alpha}))$ on $\boldsymbol{\alpha}$ from (**??**)–(**??**). $\square$

Why flatness matters. Without differential flatness, the scheduling variable would be the full n-dimensional state, and the gain Hessian $\partial^2 K/\partial x_i \partial x_j$ would be an $O(n^2)$-parameter tensor. Flatness compresses the scheduling space from $\mathbb{R}^n$ to $\mathbb{R}^4$, so the Hessian has only $\binom{4+1}{2} = 10$ independent components—making (**??**) a tight, low-dimensional bound.

Proposition 2 (Exponential time-contraction)**.** *Let $A_{\text{cl}}(\boldsymbol{\alpha}) = A_d(\boldsymbol{\alpha}) - B_d(\boldsymbol{\alpha})K(\boldsymbol{\alpha})$ be the closed-loop matrix at flat parameter $\boldsymbol{\alpha}$. The DARE solution guarantees $\bar{\rho}(A_{\text{cl}}) < 1$ (all eigenvalues strictly inside the unit circle), so the error state contracts exponentially:*

$$\|\delta x_k\| \;\le\; c\,\bar{\rho}(A_{\text{cl}})^k\, \|\delta x_0\|, \qquad c \ge 1, \qquad (25)$$

where $\delta x_k = x_k - x^*(\boldsymbol{\alpha})$, $\bar{\rho}(A_{\text{cl}}) < 1$ is the spectral radius, and c depends on the condition number of the eigenvector matrix of A_{cl}.

Contraction rate vs. constraint conservatism. For the barrier-augmented variant (Section **??**), $R_\mu = R + H_\mu(\boldsymbol{\alpha})$ increases the effective input cost, producing a less aggressive gain K_μ and a larger spectral radius $\bar{\rho}(A^{\mu}_{\text{cl}}) > \bar{\rho}(A_{\text{cl}})$. Thus the barrier parameter μ explicitly trades contraction speed for constraint margin: larger μ yields slower convergence but greater safety distance from actuator limits.

Corollary 1 (Tracking error bound)**.** *When the first-order gain approximation $\tilde{K}(\boldsymbol{\alpha})$ is used instead of the exact gain $K(\boldsymbol{\alpha})$, the closed-loop matrix becomes $\tilde{A}_{\text{cl}} = A_d - B_d\tilde{K}$. By* (**??**)*, the perturbation $\Delta K = K - \tilde{K}$ satisfies $\|\Delta K\| \le \frac{1}{2}L_K\|\delta\boldsymbol{\alpha}\|^2$. Standard perturbation theory for discrete-time systems [?] then gives*

$$\bar{\rho}(\tilde{A}_{\text{cl}}) \;\le\; \bar{\rho}(A_{\text{cl}}) + \|B_d\|\cdot \tfrac{1}{2}L_K\|\delta\boldsymbol{\alpha}\|^2 + O(\|\delta\boldsymbol{\alpha}\|^4). \quad (26)$$

The approximate closed loop remains stable whenever $\|\delta\boldsymbol{\alpha}\|$ is small enough that the right-hand side is less than 1*. For the quadrotor with $\|\delta\boldsymbol{\alpha}\| \le 3\,m/s^2$, our numerical tests (Section* **??***) confirm mean gain error of* 4% *and stable tracking across 100 random $\boldsymbol{\alpha}$ samples.*

V. Preliminary Results

We validate the proposed methods in simulation using a Crazyflie 2.1 quadrotor model (mass 35 g, arm length 46 mm) with a control frequency of 50 Hz. All simulations use a Python reference implementation with NumPy linear algebra.

A. Controller Variants

Across the three experiments we use up to five controller variants. The hover and figure-eight experiments compare four:

1) **TinyMPC** (baseline): Fixed linearization at hover, ADMM with warm-starting and convergence tolerance 10^{-3}.
2) **Online Relinearization**: Autograd-based Jacobian at each timestep, fresh DARE solve, then ADMM.
3) **Flat Template MPC**: $\boldsymbol{\alpha}$-parameterized analytic linearization with precomputed DARE cache grid, then ADMM.
4) **Analytic Template**: Zero-iteration state feedback with first-order gain interpolation.

The circle-in-square experiment (Section **??**) replaces TinyMPC with:

5) **Barrier Template**: Zero-iteration state feedback with barrier-augmented DARE cache (Section **??**), providing soft constraint awareness at ∼260 operations.

B. Hover Stabilization

The quadrotor starts at $(0.2, 0.2, -0.2)$ m from the hover point. Horizon $N = 10$, penalty $\rho = 85$, $N_{\text{sim}} = 100$ steps.

At hover, TinyMPC's fixed linearization is exact and achieves the lowest average error (Table **??**, Fig. **??**). However, it requires 10× more ADMM iterations than the Flat Template due to the high ρ value. The Flat Template converges faster because its cache already incorporates the correct equilibrium structure. The Analytic Template achieves the lowest wall-clock time (22× faster than TinyMPC) at a modest accuracy penalty.

TABLE I
HOVER STABILIZATION ($N_{\text{sim}} = 100$, $\rho = 85$, $N = 10$).

	Tiny MPC	Online Relin	Flat Template	Analytic Template
Avg. pos. error (m)	**0.064**	0.095	0.138	0.145
Max pos. error (m)	0.346	0.346	0.346	0.346
Avg. ADMM iters	80.0	16.7	7.8	0
Total ADMM iters	8002	1669	785	0
Wall time (s)	2.85	12.13	**0.44**	**0.13**

TABLE II
FIGURE-EIGHT TRAJECTORY TRACKING ($N_{\text{sim}} = 200$, $\rho = 5$, $N = 15$).

	Tiny MPC	Online Relin	Flat Template	Analytic Template
Avg. pos. error (m)	**0.033**	0.035	0.035	0.036
Max pos. error (m)	**0.091**	0.092	0.113	0.116
Avg. ADMM iters	8.0	5.1	3.6	0
Total ADMM iters	1598	1023	714	0
Wall time (s)	1.16	22.02	0.75	**0.26**

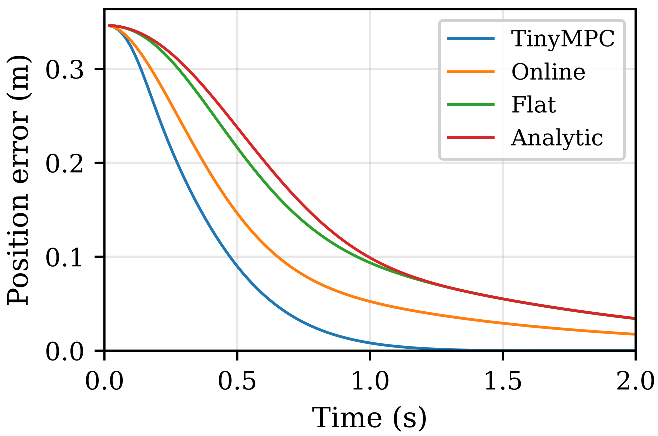

Fig. 2. Hover stabilization: position error over time for all four variants. Initial displacement $(0.2, 0.2, -0.2)$ m. All variants converge; the flat-parameterized controllers trade slightly slower convergence for significantly lower computation per step.

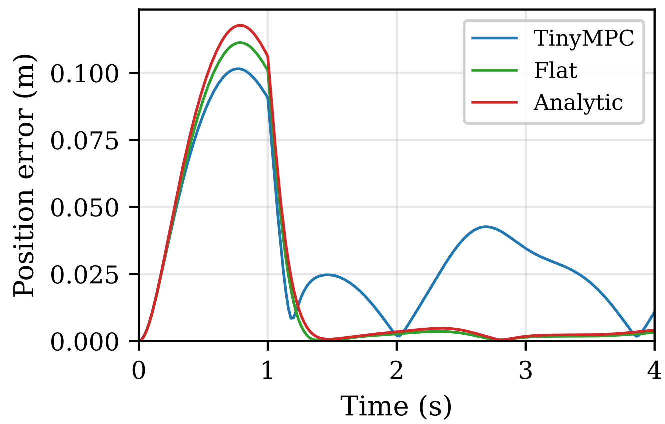

Fig. 3. Figure-eight tracking: position error over time. All variants achieve comparable accuracy; the periodic structure reflects the trajectory curvature.

C. Figure-Eight Trajectory Tracking

The reference is a figure-eight in the xz-plane with amplitude 0.5 m and period 3.7 s. Horizon $N = 15$, penalty $\rho = 5$, $N_{\text{sim}} = 200$ steps. The Flat Template uses 21 cache entries sampled uniformly over one period.

On the figure-eight trajectory (Table **??**, Figs. **??**–**??**), all four variants achieve similar average tracking errors (0.033–0.036 m), demonstrating that the flat-parameterized linearization is sufficiently accurate for moderate-aggressiveness trajectories. The key differences are computational:

- The Flat Template requires 55% fewer total ADMM iterations than TinyMPC and runs 1.5× faster in wall-clock time.
- Online relinearization is 29× slower than the Flat Template due to per-step autograd Jacobian evaluation and DARE solves, despite using fewer ADMM iterations.
- The Analytic Template achieves 4.5× speedup over TinyMPC with only 0.003 m additional average error, using *zero* ADMM iterations.

D. Circle-in-Square Constrained Tracking

To demonstrate the complexity cascade, we track a circular trajectory in the xy-plane ($R = 0.3$ m, period 8 s) while enforcing square position bounds $|x| \leq L$, $|y| \leq L$ with $L = R/\sqrt{2} \approx 0.212$ m, so that the four vertices of the square lie *exactly* on the circle (inscribed square geometry; see Fig. **??**). At the cardinal points, the circle exceeds the square by $R - L \approx 88$ mm.

We compare all four controller variants. The Online Relinearization and Flat Template both use ADMM with per-step horizon bounds: at horizon step j, the error-state position constraint is $-L - p_{\text{ref}}(t+j\Delta t) \leq \delta x_{0:2} \leq L - p_{\text{ref}}(t+j\Delta t)$, correctly mapping the absolute constraint to error coordinates at each future timestep. The Barrier Template absorbs position-constraint awareness into Q_μ via (**??**)–(**??**). The Analytic Template tracks the full circle without constraints. Parameters: $N = 20$, $\rho = 50$, up to 500 ADMM iterations (for ADMM variants).

Table **??** and Fig. **??** illustrate the central tradeoff of the hierarchy (**??**). The two ADMM variants (Online Relinearization and Flat Template) achieve near-zero position violation by encoding the absolute constraint as per-step error-state bounds along the prediction horizon, actively projecting the trajectory onto the feasible set at each ADMM iteration. The Flat Template achieves the same constraint satisfaction as Online Relinearization while eliminating the per-step DARE solve.

The zero-iteration variants (Barrier and Analytic) exhibit the full geometric violation of $R - L = 88$ mm. This is expected: the barrier Hessian (**??**) penalizes error-state excursions from the reference, but in this geometry the *reference itself* exceeds the square boundary at the cardinal points ($R > L$). No modification of the error-state cost can prevent absolute position violation when the tracking target lies outside the feasible

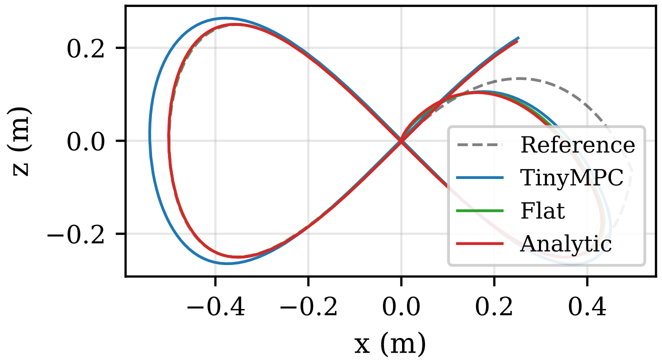

Fig. 4. Figure-eight tracking: x–z plane view. The Flat Template and Analytic Template closely follow the reference trajectory (dashed), comparable to TinyMPC.

TABLE III

CIRCLE-IN-SQUARE TRACKING ($R = 0.3$ M, $L = R/\sqrt{2}$, PERIOD 8 S). MOVING DOWN THE CASCADE TRADES CONSTRAINT FIDELITY FOR REAL-TIME GUARANTEES.

	Online Relin	Flat Templ. (ADMM)	Barrier (zero-iter)	Analytic (zero-iter)
Complexity	$O(n^3{+}Kn^2)$	$K{\cdot}O(n^2)$	$O(nm)$	$O(nm)$
Constraints	Hard	Hard	Soft	Clip
Avg. pos. err (m)	0.062	0.062	0.017	0.017
Max viol. (mm)	0.0	0.0	88.4	88.4
Mean viol. (mm)	0.0	0.0	45.8	45.7
Avg. ADMM iters	450.8	450.8	0	0
Wall time (s)	168.1	157.4	0.65	0.62

region—only the ADMM projection (which maps absolute bounds to per-step error-state bounds) can achieve this. The barrier remains effective for *input* constraints (preventing motor saturation) and for state constraints where the reference stays inside the feasible region.

The key message: *moving down the cascade trades constraint fidelity for real-time guarantees*. The ADMM variants handle arbitrary absolute constraints via iterative projection at $O(n^2)$ per iteration; the zero-iteration variants achieve deterministic $O(n{\cdot}m)$ feedback ($\sim$260 operations) enabled by the flatness-compressed $\mathbb{R}^4$ scheduling space, at the cost of relinquishing hard state-constraint satisfaction.

E. Numerical Validation

We verify the implementation with four tests:

1) **Linearization consistency**: At hover, the analytic A_d, B_d match the autograd-based Jacobians to within 10^{-3} in position-velocity coupling and 10^{-15} in torque allocation. Attitude-related entries differ by $\sim 2\times$ due to the Euler-angle vs. quaternion-error parameterization.
2) **Gain approximation**: Over 100 random $\boldsymbol{\alpha}$ samples with $\|a\| \le 3\,\mathrm{m/s^2}$, the first-order gain approximation (**??**) achieves mean relative Frobenius error of 4%.
3) **Control equivalence**: At hover with inactive constraints, the Analytic Template output equals the 1-iteration Flat Template output to machine precision ($< 10^{-8}$).

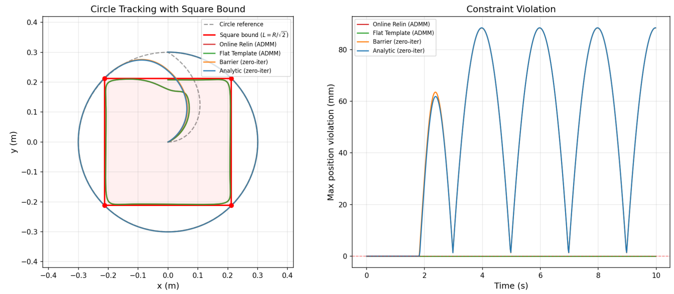

Fig. 5. Circle-in-square constrained tracking: the complexity cascade from $O(n^3)$ online NMPC to $O(n{\cdot}m)$ deterministic feedback. **Left:** xy-plane trajectories. Online Relin and Flat Template (ADMM) are visibly "pinched" toward the square at the cardinal points; both zero-iteration variants (Barrier and Analytic) track the full circle with identical 88 mm violation, confirming that the barrier Hessian cannot override a reference that lies outside the feasible region. **Right:** Position violation over time, illustrating the constraint-fidelity/computation tradeoff.

4) **Flat equilibrium**: For 20 random $\boldsymbol{\alpha}$ values, $f(x^*(\boldsymbol{\alpha}), u^*(\boldsymbol{\alpha}))$ produces the expected continuous-time derivatives to $< 10^{-14}$.

VI. DISCUSSION

When does the fixed linearization suffice? For near-hover operation, TinyMPC's fixed linearization is optimal by construction. The flat-parameterized variants become advantageous when the trajectory induces significant attitude variation—the equilibrium shift ensures the ADMM solver operates near zero error at every timestep, improving convergence.

Computational hierarchy. The five variants form a hierarchy of decreasing online cost (see also the equivalence chain (**??**)):

Variant	Online ops	Constr.	Conv.	Sched.
Online Relin	$O(n^3){+}KO(n^2)$	Hard	Lin.	$\mathbb{R}^n$
Flat Template	$K \cdot O(n^2)$	Hard	Lin.	$\mathbb{R}^4$
Barrier + IPM	$K_\mathrm{I} \cdot O(n^2)$	Hard	Quad.	$\mathbb{R}^4$
Barrier	$O(n \cdot m)$	Soft	—	$\mathbb{R}^4$
Analytic	$O(n \cdot m)$	Clip	—	$\mathbb{R}^4$

The central contribution is the path from cubic-time online optimization to linear-time deterministic feedback, enabled by flatness-compressed scheduling over $\mathbb{R}^4$. The Flat Template has the same per-iteration cost as TinyMPC but converges in fewer iterations due to better-conditioned linearization. The Barrier Template (Section **??**) provides soft constraint satisfaction at the same $\sim$260-operation cost as the Analytic Template, by absorbing log-barrier information (both H_μ for inputs and $H_{x,\mu}$ for states) into the DARE cache. With optional IPM refinement (2–3 Newton steps), it achieves hard constraint satisfaction with quadratic convergence—fewer iterations than ADMM for the same accuracy. The Analytic Template ($\mu = 0$) trades all constraint handling for extreme efficiency. The zero-iteration variants (Barrier and Analytic) achieve real-linear-time control: deterministic, iteration-free, $O(n{\cdot}m) \approx 260$ operations per control output, only possible because flatness compresses the scheduling space to $\mathbb{R}^4$ (4 sensitivity matrices, 240 floats).

What is new vs. what is classical. The individual mathematical ingredients—differential flatness [**?**], [**?**], gain scheduling [**?**], and Riccati sensitivity [**?**], [**?**]—are well established. Our contribution is their specific combination in the context of embedded ADMM-based MPC: (i) using flatness to compress the scheduling dimension from n to 4, (ii) caching the *augmented* DARE (with ADMM penalty ρ) rather than the standard LQR DARE, (iii) showing the equivalence between the zero-iteration ADMM forward pass and the flat gain schedule, and (iv) absorbing log-barrier constraint penalties into the DARE cache so that the zero-iteration controller is inherently constraint-aware (Section **??**). This combination yields the 240-float, 260-operation datapath of the Analytic Template—a level of compactness not achievable by generic gain scheduling or LPV approaches without the flatness reduction—while the barrier augmentation adds constraint awareness at no additional online cost.

FPGA portability. The analytic linearization (Section **??**) and the Analytic Template (Section **??**) are composed entirely of elementary operations: trigonometric functions, multiplications, and additions. There are no loops with data-dependent iteration counts and no matrix inversions. The 240-float storage and $\sim$260-operation datapath are directly realizable as a fixed-function pipeline.

Limitations. The first-order gain approximation (**??**) degrades for large $\|\boldsymbol{\alpha} - \boldsymbol{\alpha}_0\|$. For highly aggressive maneuvers, the Flat Template MPC with a sufficiently dense cache grid is preferred. All wall-clock times are from a Python/NumPy reference implementation and reflect relative computational cost rather than embedded performance; C/C++ implementation with Crazyflie hardware experiments is ongoing work.

VII. Conclusion

We presented three extensions of TinyMPC that exploit differential flatness to adapt the linearization point along a trajectory. The Flat Template MPC precomputes a family of DARE caches indexed by the flat parameter $\boldsymbol{\alpha}$, preserving TinyMPC's matrix-inversion-free online structure while achieving faster ADMM convergence. The Analytic Template further reduces the online computation to $\sim$260 arithmetic operations by replacing ADMM with a first-order gain interpolation. The Barrier-Augmented Template absorbs log-barrier constraint penalties into the DARE cache, providing inherent constraint awareness at zero additional online cost, with optional Newton–IPM refinement for hard constraint satisfaction at quadratic convergence. Preliminary simulation results demonstrate that the flat-parameterized variants maintain tracking accuracy comparable to online relinearization at substantially lower computational cost. Future work includes C implementation targeting the Crazyflie 2.1 STM32F405 microcontroller, FPGA synthesis of the Analytic Template datapath, numerical validation of the Barrier-Augmented Template, and extension to wind-disturbance rejection.

References

[1] A. Alavilli, K. Nguyen, S. Schoedel, B. Plancher, and Z. Manchester, "TinyMPC: Model-predictive control on resource-constrained microcontrollers," in *Proc. IEEE Int. Conf. Robotics and Automation (ICRA)*, 2024.

[2] D. Mellinger and V. Kumar, "Minimum snap trajectory generation and control for quadrotors," in *Proc. IEEE Int. Conf. Robotics and Automation (ICRA)*, 2011, pp. 2520–2525.

[3] R. M. Murray, M. Rathinam, and W. Sluis, "Differential flatness of mechanical control systems: A catalog of prototype systems," in *Proc. ASME Int. Mechanical Engineering Congress*, 1995.

[4] M. W. Mueller, M. Hehn, and R. D'Andrea, "A computationally efficient motion primitive for quadrocopter trajectory generation," *IEEE Trans. Robotics*, vol. 31, no. 6, pp. 1294–1310, 2015.

[5] M. Fliess, J. Lévine, P. Martin, and P. Rouchon, "Flatness and defect of non-linear systems: introductory theory and examples," *Int. J. Control*, vol. 61, no. 6, pp. 1327–1361, 1995.

[6] B. Stellato, G. Banjac, P. Goulart, A. Bemporad, and S. Boyd, "OSQP: An operator splitting solver for quadratic programs," *Mathematical Programming Computation*, vol. 12, pp. 637–672, 2020.

[7] S. Boyd, N. Parikh, E. Chu, B. Peleato, and J. Eckstein, "Distributed optimization and statistical learning via the alternating direction method of multipliers," *Foundations and Trends in Machine Learning*, vol. 3, no. 1, pp. 1–122, 2011.

[8] M. Faessler, A. Franchi, and D. Scaramuzza, "Differential flatness of quadrotor dynamics subject to rotor drag for accurate tracking of high-speed trajectories," *IEEE Robotics and Automation Letters*, vol. 3, no. 2, pp. 620–626, 2018.

[9] W. J. Rugh and J. S. Shamma, "Research on gain scheduling," *Automatica*, vol. 36, no. 10, pp. 1401–1425, 2000.

[10] J. S. Shamma and M. Athans, "Guaranteed properties of gain scheduled control for linear parameter-varying plants," *Automatica*, vol. 27, no. 3, pp. 559–564, 1991.

[11] G. A. Hewer, "An iterative technique for the computation of the steady state gains for the discrete optimal regulator," *IEEE Trans. Automatic Control*, vol. 16, no. 4, pp. 382–384, 1971.

[12] M. M. Konstantinov, P. Hr. Petkov, and N. D. Christov, "Perturbation analysis of the discrete Riccati equation," *Kybernetika*, vol. 29, no. 1, pp. 18–29, 1993.

[13] A. Bemporad, M. Morari, V. Dua, and E. N. Pistikopoulos, "The explicit linear quadratic regulator for constrained systems," *Automatica*, vol. 38, no. 1, pp. 3–20, 2002.

[14] S. Boyd and L. Vandenberghe, *Convex Optimization*. Cambridge Univ. Press, 2004.

[15] A. Jordana, S. Kleff, A. Meduri, J. Carpentier, N. Mansard, and L. Righetti, "Stagewise implementations of sequential quadratic programming for model-predictive control," in *Proc. IEEE/RSJ IROS*, 2023.

[16] J. Zhang, "TTT: Tunnel, Threshold, and Turbulence—a unified framework for iterative optimal control," *Preprint*, 2026.

[17] J. Zhang, "Yang–Mills meets optimal control: renormalization group structure of the DARE iteration," *Preprint*, 2026.

[18] J. Zhang, "ABC of constrained optimal control: adjoint, barrier, and controllability," *Preprint*, 2026.

[19] G. Williams, N. Wagener, B. Goldfain, P. Drews, J. M. Rehg, B. Boots, and E. A. Theodorou, "Information theoretic MPC for model-based reinforcement learning," in *Proc. IEEE Int. Conf. Robotics and Automation (ICRA)*, 2017, pp. 1714–1721.

Appendix

This appendix situates the zero-iteration controller within a broader theoretical landscape, drawing connections to three companion analyses: the tunnel–threshold–turbulence (TTT) framework for iterative optimal control [**?**], the renormalization-group (RG) interpretation of the DARE iteration [**?**], and the adjoint–barrier–controllability (ABC) analysis of constrained optimal control [**?**]. These connections explain *why* the barrier-augmented zero-iteration controller cannot enforce absolute state constraints and suggest concrete paths beyond this limitation.

A. The Zero-Iteration Controller as a Seed

The zero-iteration control law $u = u^*(\boldsymbol{\alpha}) - K(\boldsymbol{\alpha})\,\delta x$ is a *seed* in the sense of [**?**]: a fixed-point controller at one operating point (the reference trajectory), extended to nearby points by first-order sensitivity.

TTT's main theorem establishes four equivalent conditions:

$$\text{bounded } P_j \iff \text{spectral gap} > 0 \iff \text{Lyapunov descent} \iff \text{tunnel exists.} \tag{27}$$

Our DARE solution P_∞ is the bounded cost-to-go guaranteed by the first condition. The "tunnel" is the sublevel set $\mathcal{T} = \{x : \frac{1}{2}x^\top P_\infty x \le V_{\max}\}$ inside which the seed controller maintains exponential contraction—the region of attraction whose radius scales as $r \sim \|P_\infty\|^{-1/2}$.

From the Yang–Mills perspective [**?**], the DARE iteration $P_{j+1} = f(P_j)$ is a discrete *renormalization group flow*: each step integrates out one "scale" of the control problem (one stage of the finite-horizon cost). Convergence of $P_j \to P_\infty$ is the analogue of the mass gap in Yang–Mills theory—the spectral gap between the largest and second-largest eigenvalue of the closed-loop transition matrix. The contraction rate $\bar{\rho}(A_{\mathrm{cl}}) < 1$ (Proposition **??**) is precisely this gap.

B. Why Zero-Iteration Cannot Enforce Absolute Constraints

The circle-in-square experiment (Table **??**) reveals that the barrier-augmented controller violates the square bound by the same 88 mm as the unconstrained analytic controller. We now explain this result through three complementary lenses.

Costate rotation (ABC). The gain $K(\boldsymbol{\alpha})$ is the costate ∇V evaluated at the reference (equilibrium) point [**?**]. The full costate *field* $\nabla V(x)$ varies spatially: near a constraint boundary, ∇V rotates to point away from the boundary, encoding the directional information needed to steer the trajectory into the feasible region. A constant-gain approximation $K \approx \nabla V(x^*)$ cannot capture this rotation, regardless of the barrier parameter μ. The barrier Hessian (**??**) modifies the *magnitude* of the gain (increasing the effective state cost near constraints) but not its *direction*—and it is the directional information that determines constraint satisfaction when the reference lies outside the feasible region.

UV–IR decoupling (Yang–Mills). In the language of [**?**], the barrier modifies the "UV source term" Q_μ (the error-state cost in the DARE) but cannot create "IR contraction" (the iterative projection that enforces absolute constraints over the prediction horizon). The zero-iteration controller operates entirely in the UV regime—local, single-point, based on the equilibrium Hessian. Hard constraint satisfaction is an IR property: it requires the full cost-to-go landscape $V(x)$, not just its Hessian $\nabla^2 V(x^*)$ at one point. Both regimes must cooperate—UV provides the seed, IR provides the tunnel.

Seed vs. tunnel (TTT). The seed alone does not constitute a tunnel. The tunnel is constructed by iterative refinement (ADMM iterations, in our setting) that progressively tightens the constraint satisfaction. This is precisely what the ADMM variants achieve via per-step horizon bounds (Section **??**): each iteration projects the predicted trajectory toward the feasible set, building the tunnel one iteration at a time. The zero-iteration controller, by definition, performs no such construction.

C. Beyond Zero-Iteration: Path Integral Extensions

Three paths to constraint-aware real-time control, ordered by computational budget:

1. MPPI-seed hybrid ($O(N_s \cdot n) \approx 1500$ ops). Use $K(\boldsymbol{\alpha})$ as the base policy and sample $N_s \approx 10$ perturbations around the seed trajectory. Weight each sample by the Boltzmann factor $e^{-\beta \cdot J}$ where J includes a constraint penalty [**?**]. The weighted average creates a soft "tunnel" via ensemble averaging: samples that violate constraints receive low weight. This is deterministic-time (no convergence loop) and respects the TTT structure: the seed provides the center of the sampling distribution, while the Boltzmann weighting constructs a statistical approximation to the tunnel boundary.

2. Costate lookup table ($O(n \cdot m)$ + table lookup ≈ 500 ops). Precompute the costate field $\nabla V(x)$ at a grid of positions near constraint boundaries. Online, look up the nearest costate vector and correct $K(\boldsymbol{\alpha})$ with the directional information. This adds ABC's "missing ingredient"—the costate *field* rather than a single-point costate—to the zero-iteration controller. The lookup table size scales with the number of constraint boundaries, not the state dimension, keeping the approach compact for box-constrained systems.

3. Constraint-shaped terminal cost ($O(n \cdot m) \approx 260$ ops, same budget as the Analytic Template). Instead of adding the barrier Hessian at the equilibrium, design P_∞ via a DARE whose terminal cost encodes the constraint geometry along the predicted trajectory. This requires a one-step prediction $x_{k+1} = A_d x_k + B_d u_k$ before the gain lookup but otherwise preserves the $\sim$260-operation datapath. The key difference from the current barrier approach: the terminal cost is evaluated along the trajectory, not at the equilibrium—providing the "IR information" that the equilibrium-only barrier lacks.

These three extensions correspond to TTT's nested game structure [**?**]: the MPPI-seed hybrid (outer game, sampling/exploration), costate lookup (middle game, adjoint/directional information), and constraint-shaped terminal cost (inner game, cost landscape shaping). The tunnel emerges from the cooperation of all three mechanisms, not from any single one—mirroring the experimental finding that no single zero-iteration mechanism (barrier, clipping, or gain interpolation) suffices for hard constraint satisfaction.